TravelPort Software Requirements Specification

Purpose and scope

TravelPort is a responsive travel-booking platform for customers, administrators, hotel managers, and transport operators. It supports flight, hotel, bus, train, and cab discovery; booking and cancellation; payments and wallets; invoices; notifications; and optional AI-assisted planning.

User roles

- Customers register, authenticate, search inventory, create and cancel bookings, manage travellers/cards, use coupons and wallets, and download invoices.
- Administrators manage users, catalogue data, coupons, bookings, announcements, and reporting.
- Hotel managers manage their assigned hotel, rooms, availability, reviews, and bookings.
- Flight, bus, and cab operators manage only records belonging to their assigned company.

Functional requirements

1. The system shall issue short-lived JWT access tokens and renewable refresh tokens after successful authentication.
2. The system shall enforce role and resource ownership on protected endpoints.
3. Search shall validate routes, dates, passenger/guest counts, and availability.
4. Booking shall calculate totals and discounts, preserve traveller/contact details, and return a unique booking reference.
5. Cancellation shall enforce status rules and record any approved refund.
6. Wallet and payment operations shall be transactional and auditable.
7. Invoice endpoints shall generate readable flight and hotel PDF documents.
8. Optional Groq, Amadeus, Duffel, Razorpay, and SMTP integrations shall degrade safely when disabled or unconfigured.
9. Fresh public deployments shall seed catalogue data only and shall not create users, passwords, wallets, or bookings.

Data and deployment requirements

- SQL Server 2019 or newer is required.

- The SSDT project and its DACPAC are the authoritative shared-environment deployment mechanism.
- The DACPAC shall be published successfully before API deployment.
- EF Core migrations shall remain synchronized for application mapping and explicit local-development fallback only.
- Secrets shall be supplied through ignored local files, environment variables, user secrets, or a company-approved secret store.

Quality requirements

- Backend and frontend builds, tests, lint, dependency audits, secret scans, DACPAC build/publish validation, and Docker Compose validation must pass before release.
- API failures shall return consistent problem details without exposing secrets or stack traces.
- Passwords shall be BCrypt-hashed; full payment card numbers shall never be stored.
- The web application shall support current desktop and mobile browsers.

Acceptance criteria

A fresh clone following the root README can build, configure the database, start the API and frontend, register a new customer, complete representative search/booking flows, and generate valid invoices without relying on public default credentials.